

What should you look for in a chip?

Which chip do you select for high-performance parallel calculations and control applications? How do you reduce the power consumption on a chip? Satish Bagalkotkar, president and CEO, and Devesh Gautam, COO, Synapse Design answer the above questions and more as they speak with Dilin Anand of EFY at Synapse's new office in Whitefield

What are the most significant technological advances that enable reducing power consumption in a chip?

They are advanced grain power gating, fine grain clock-gating techniques, RTL modification for clock gating (EDA tool based) and power-aware architecture of the RTL (RTL structures, data path parallelisation and optimisation of the code in terms of VT usage). Overall, we want to reduce the toggle rate of heavy load nodes with RTL changes or special custom cell design. In a nutshell, design for multipower domain based on an application instead of the whole circuit all the times.

What are the latest techniques for getting the chip to work at a lower power level in the real world?

Before we start most low-power designs, we review the product specifications and physical cell libraries and memory macros. This is to reduce library cells that use deep stacks and memory macros with small timing and voltage margin.

Since you have an expertise in CPU and GPU architectures, could you give us examples of the applications where a GPU would be a better choice than a CPU?

GPUs are optimal for tight code with high parallelisation. They have hundreds of simpler execution units with single instruction, multiple data (SIMD) capabilities that enable algorithms that can be parallelised easily to be best implemented on GPUs. At the same time, the GPU normally has smaller memory availability in normal implementation than a CPU, so a GPU is probably not well suited for a data-intensive application.

What makes the GPU better at the applications mentioned above than a CPU?

Applications listed above produce a lot of data that have to be processed very fast. GPU is mainly used as an accelerator for SIMD processing and CPU is used as a controller. Architecturally, most of the GPU blocks (shader and rendering engine) are used in parallel for faster operation.

What is the primary difference in working between a CPU and a GPU—with respect to the analogy that a CPU is like being an executive while a GPU is a labourer?

Most CPUs have an execution unit with branch and loops capability that is used for controlling other logic blocks. As an



analogy, we can equate CPU as an executive or boss and cores as labourers. GPUs are meant for highly parallel calculation, where data streams are operated upon, not for control application like a normal CPU is.

What should you look for in a chip?

Some features that apply to many industries are low power and performance, design of testability and manufacturability, hardened blocks for best speed, area and power (PPA)—ARM processor, GPU shader, network processor core, security engine and interface blocks—PCIe and SerDes and mixed signal design.

Since you are also into FPGA, does the traditional FPGA value proposition of reduced time-to-market, lower development cost (total) and flexibility completely apply in the test and measurement world too?

I personally think it does, because it allows the smaller production runs that some instrumentation companies need. However, speed and capacity are still a problem. Very high performance scopes, for example, have a data throughput need that is simply not compatible with FPGAs. I once worked on an ASIC for a scope that needed 1.5Gbit of embedded memory, with a total throughput of 200Gbps in and out of the chip or one that requires complex filters and FFT processing.

What approach is taken by SoC designers to tackle the greater share of analogue/mixed-signal circuitry in the chip—especially since analogue does not scale as well as digital.

At the block level, we enforce a methodology using thorough transistor level simulation at all the PVT corners. We use silicon-verified sub-blocks as much as possible, implement the design for testability for analogue blocks and verify all the block-level interfaces between analogue and digital circuits. A thorough design review is held with the customer and block owners.

In verification, we help our customers with writing the verification models and implementing an efficient verification methodology, such as AMS Verilog. ●